

Static Type Inference for Optimized Lua Program Execution in Parallel and Distributed Architectures

Carlos Henrique C. Duarte^{1,*}

Brazilian Development Bank (BNDES), Rio de Janeiro, RJ, Brazil

Paulo Rogério da Motta Júnior

National Laboratory for Scientific Computing, Petrópolis, RJ, Brazil

Abstract

Type systems for programming languages are usually formulated to ensure runtime safety and optimization, and for abstraction, modularization and documentation purposes. However, not every programming language is designed with built-in type systems and typing algorithms. It is possible to reconcile the views behind typed and untyped languages by adopting optional typing schemes and tools. In this way, programmers can recover type-safety and runtime optimization while effectively and transparently using the resources available in modern computing architectures, such as multiple CPUs, GPUs and VPUs.

This paper defines an optional type system that integrates parametric polymorphism with subtyping, a static type inference algorithm and a code transformation scheme suitable for the optimized execution of Lua programs in parallel and distributed architectures. We show that subtyping is deterministic, and type inference is decidable in this system. Moreover, we argue that the proposed code transformation scheme is correct with respect to preserving the language semantics. We also present some examples and analyses based on the tools we have implemented, which provide empirical evidence of the obtained performance improvements.

Keywords: Lua Programming Language, Parallel and Distributed Computing, Static Type Inference, Subtyping, Type Systems.

*Corresponding author

Email addresses: carlos.duarte@computer.org (Carlos Henrique C. Duarte), prmottajr@gmail.com, prmottajr@acm.org (Paulo Rogério da Motta Júnior)

¹The assumptions, views and opinions in this paper are solely those of the author and do not necessarily reflect the official policy, strategy or position of any Brazilian government entity. Carlos Henrique is on leave of absence from BNDES in a temporary work assignment at the Brazilian Institute of Statistics and Geography (IBGE).

1. Introduction

Type systems for programming languages are usually proposed to ensure runtime safety and optimization, and for abstraction, modularization and documentation purposes. In particular, native static type verification and inference procedures, formulated together with the definition of high-level programming languages, can avoid runtime type mismatch errors and the overhead of performing runtime type verification.

However, not every programming language is designed with built-in static type systems and respective type verification algorithms. For example, Scheme [1], Python [2], Lua [3], JavaScript [4] and Ruby [5] were not originally designed with type systems and are not statically typed, but some of these languages perform dynamic type checking during program execution. The main reasons for this design decision were to ensure programming flexibility [6], to avoid early program rejection [2] and to prevent stopping program execution prematurely [5].

It is possible to reconcile the views behind typed and untyped programming languages by specifying and implementing type systems that are not of mandatory usage, so-called optional typing schemes [6]. With an optional type system, programmers can deal with typing information in programs only if they desire to do so, but in these cases they can benefit from the type-safety and runtime optimizations delivered by tools that are of optional usage while ensuring the effective and transparent use of available resources.

In particular, it is possible to address the challenge to develop programs that correctly and efficiently execute in parallel and distributed architectures using optimal typing schemes [7, 8]. By allowing explicit type annotations in programs, relying on type inference procedures and adopting specific compile-time tools that abstract away any target execution infrastructure, but ensure the generation of efficient code, it is possible to take advantage of the resources available in modern computing architectures, such as multiple CPUs, GPUs and VPUs, without being exposed to the complexity of the underlying runtime platform.

This paper defines a type system that integrates parametric polymorphism with subtyping, a static type inference algorithm and an optimizing code transformation scheme suitable for executing Lua programs in parallel and distributed architectures. We show that subtyping is deterministic, and type inference is decidable in this system. Moreover, we argue that the proposed code transformation scheme is correct with respect to preserving the language semantics. We also present some examples and analyses based on the tools we have implemented, which provide empirical evidence of the obtained performance improvements.

Although other optional typing schemes for Lua have been proposed in the literature [9, 10], our work is innovative due to the connection with the optimized execution of Lua programs in parallel and distributed architectures while preserving the original Lua language definition. Indeed, it is precisely in terms of the standard Lua definition that our type system is specified, and subtyping determinism and type inference decidability are verified. More generally, we believe that our research contributes to facilitating the execution of optionally typed programming languages in parallel and distributed architectures.

This paper is organized as follows. Section 2 describes the Lua language. Section 3 discusses an implementation strategy for efficiently executing Lua programs in multicore architectures. Section 4 defines a formal type system with subtyping, a static type inference algorithm and a code transformation scheme suitable for Lua programs. We present in Section 5 some examples and empirical analyses based on the prototypes we have implemented. Finally, we compare our results to related work in Section 6 and conclude the paper by discussing our findings and presenting prospects for future research (Section 7).

2. A (Very Brief) Overview of the Lua Language

Lua is a high-level language that supports a scripting style of programming [11]. It is particularly suitable for embedding interpreted applications into host C programs. The standard distribution of the language consists of a standalone interpreter and a library exposing a C API.

Lua has a procedural syntax and powerful data abstraction mechanisms. The standard control structures are all present in the language, such as `if then else`, `while do`, `repeat until` and `for`. It has numerical and relational overloaded operators, supports user-defined dynamically allocated data structures, higher-order polymorphic functions and other data abstraction mechanisms. In this way, Lua supports both parametric and ad-hoc polymorphism [12].

Concerning the adopted typing scheme, Lua is weakly dynamically typed. This means that each value carries implicit typing information, and so the runtime environment performs any required type conversion. Despite the support to user-defined data structures, Lua does not allow type declarations, nor programmer-defined structured types, since typing mechanisms are not present in the language.

Lua has four basic primitive data types: booleans, numbers, strings and the type corresponding to the distinguished value `nil`. Numbers correspond to integer or floating-point data. Strings are immutable sequences of bytes. The language also supports structured objects: tables, functions, threads and user data. In particular, tables are associative arrays that can be used to simulate structured objects. The respective types are introduced in Section 4.

Although Lua has a compact definition, the language semantics is based on meta-level features, which are beyond the reach of standard first-order type systems. For example, it is possible to use meta-variables and meta-functions in Lua programs to customize the semantics of some language constructs. We do not address these features in this paper and restrict our research to what we call Plain Lua here.

3. A (Simple) Strategy for Concurrently Executing Lua Programs

The availability of multiple computing cores in modern computing architectures points out the need for programming language tools that effectively use such resources in safe and efficient ways. The current interest in using VPUs for scientific computing and GPUs for computer games and graphical computing are particular examples of this situation. However, integrating GPUs and VPUs in parallel and distributed computing platforms is a challenging task since their structure and functionality are different from those of CPUs. Consequently, any approach to dealing with multicore architectures must ensure the simple and easy usage of available resources while ensuring improved safety and performance.

Our strategy to deal with this problem is to support the separate compilation of parallelizable functions present in the source code into executable code. In order to implement this strategy, we presume the existence of a modular architecture for parallel and distributed computing. The main components of this reference architecture are called a Source-to-Source Translator, a Compiler Provider and a Device Manager, an extension of the model first proposed in [7]. The Source-to-Source Translator performs type inference and code transformation routines, such as identifying and separating functions to be parallelized. The Compiler Provider is responsible for the separate compilation of the selected functions. Finally, the Device Controller offers architecture-dependent functionalities, such as providing information on the available runtime resources, loading the generated code on each device, performing optimizations, providing function parameters and retrieving obtained results.

According to this model, the original and transformed programs serve as means of interaction between pairs of architectural components. These interactions are relatively transparent to the application programmer and happen abstractly, hiding hardware and operating system details and ensuring the safe and effective usage of available resources. However, these benefits can only be obtained if function parameter and return types are determined in advance. That is why static type inference is a central requirement of this model. It ensures the provision of the required typing information in advance, without forcing the specification of any additional information in the source code.

4. A Type System for Plain Lua

The Lua language was not designed with a formally defined type system, so the definition of appropriate typing procedures is a challenging task. Indeed, Lua operators are overloaded, and functions may be high-order, polymorphic and recursive. Moreover, parameter passing and structured object assignments are allowed only by reference. Therefore, the definition of a type system for Plain Lua requires type variables and subtyping, and sequence and union types.

4.1. Type Expressions

Lua is rigorously defined in [11]. The language grammar is specified using a set of production rules, which we call σ here. We denote by $\mathcal{L}(\sigma)$ the language generated by σ and by $\mathcal{C}(\sigma) = \{f \in \mathcal{L}(\sigma') \mid \sigma' \subseteq \sigma\}$ the set of Lua code fragments.

Recall that Lua expressions evaluate to values, which are all implicitly bound to types. Due to this fact, we adopt a flat type system in which values and types are not mixed and higher-order structures, such as Smalltalk meta-classes [13] and Haskell type classes [14], are not allowed.

We use additional production rules to introduce type tags in the language. The first rule treats basic primitive types (τ_s), whereas the other ones address additional basic numeric (τ_n) and structured object (τ) types. The definition of basic types is presented below:

$$\tau_b ::= \text{Nil} \mid \text{Boolean} \mid \text{String} \mid \text{Number} \quad (1)$$

In order to provide support to the specification of explicit and detailed numerical type information in programs, something useful in code transformation and generation, we define specialized numeric type tags (in addition to Number , which is already defined in τ_b):

$$\tau_n ::= \text{Int} \mid \text{Long} \mid \text{Single} \mid \text{Double} \quad (2)$$

These specialized numerical type tags comply with the Lua language definition [11]. They allow two distinct but mutually convertible internal number representations called *Integer* and *Float*, defined as abbreviations here. Integer type tags, on the one hand, correspond to 32-bit (*Int*) or 64-bit (*Long*) representations. On the other hand, *Single* denotes 32-bit floating-point numbers, whereas *Double* denotes a 64-bit float representation.

We presume the existence of two unbounded but denumerable sets of variables: one for type (Ξ) and another one for program (Ψ) variables. We use a generic t to denote types, and $u \in \Xi$ and $v \in \Psi$ to represent type and program variables. Thus, we write $v : t$ (or u) to mean that the value denoted by v has type t (or u).

We use the symbol $*$ as a superscript to denote sequences of types. Sequences are useful in the definition of the production rule for type expressions:

$$\tau ::= \tau_b \mid \tau_n \mid u \mid \text{Thread} \mid \text{UserData} \mid (\tau \mapsto \tau)^* \mid \tau^* \rightarrow \tau^* \mid \cup \tau^* \mid \forall u \cdot \tau \mid \text{Any} \quad (3)$$

Some clarification regarding (3) is required here since this production rule keeps a loose connection with concrete Lua language constructs:

- Thread is the type adopted in the implementation of coroutines (cf. [15]);
- UserData denotes the type of externally defined user data, such as those in embedding C programs;
- The type of tables is defined in terms of sequences of associations, each one denoted by $\tau \mapsto \tau$;
- The type of functions, denoted by $\tau^* \rightarrow \tau^*$, has domain and codomain ranging over sequences of types, since Lua functions can receive and return varying numbers of values;
- Unions of type sequences represent variable value types, thus capturing ad-hoc polymorphism [12];
- Universally quantified types are used to deal with first-order type schemes and generic functions [16], allowing us to capture parametric polymorphism;
- Any denotes the top element of our type hierarchy;

We have to introduce some notational conventions due to our synthetic definition of type expressions. We use the $\{\dots\}$ notation to represent both tables and their types. Concerning the type of sequences, we interchangeably adopt the $\langle \dots \rangle$ notation to represent sequences defined by enumeration and the $:$ notation to represent those defined in terms of their heads and tails.

Sequences and unions are used as in [1] for representing usual type constructs. For instance, we define the bottom type $\perp \stackrel{\text{def}}{=} \cup \langle \cdot \rangle$ as the empty sequence union. Note that the following equalities hold for each type t : $\cup \langle t \rangle = \langle t \rangle = t$. Concerning numeric types, we define $\text{Integer} \stackrel{\text{def}}{=} \text{Int} \cup \text{Long}$ and $\text{Float} \stackrel{\text{def}}{=} \text{Single} \cup \text{Double}$. We use τ_a to denote the set containing all the previously mentioned type tags. Moreover, concerning object types, we define $\text{Table} \stackrel{\text{def}}{=} \{ \cdot \}$ and $\text{Function} \stackrel{\text{def}}{=} \perp \rightarrow \text{Any}$, the most general elements of the respective types. Finally, concerning object types, $\text{Object} \stackrel{\text{def}}{=} \text{Thread} \cup \text{UserData} \cup \text{Table} \cup \text{Function}$. These definitions and the subtyping relationships described in Section 4.2 lead our type hierarchy to have the shape presented in Fig. 1.

The lattice structure in Fig. 1 corresponds to the chosen algebraic structure to support the assignment of identifiers and expressions to types. Because sequences cannot be assigned to program variables in Lua, they do not appear individually in the figure nor in rule (3). Despite this, some of our formal constructions deal with intermediate values of this type. Alternatively, we could have chosen a more compact algebraic structure, considering that the Lua language implementation treats strings as a particular kind of sequence and sequences as a specific kind of table [11]. Nevertheless, the structure in Fig. 1 appears to us as the most closely related to the Lua language high-level definition. We find it convenient to have a bottom and a top element in the type system because they are helpful in the formulation of our definitions. Finally, the absence of referential types among our definitions should not surprise the reader since parameter passing and object assignment in Lua generate aliases, which are transparently treated (dereferenced).

Due to our goal to allow the optional introduction of typing information in Lua programs, we extend the original language grammar with the production rules above. We thus generate a new grammar σ_τ and language $\mathcal{L}(\sigma_\tau)$ for optionally typed Lua programs. In addition, we consider that any free type variable in a program is implicitly universally quantified.

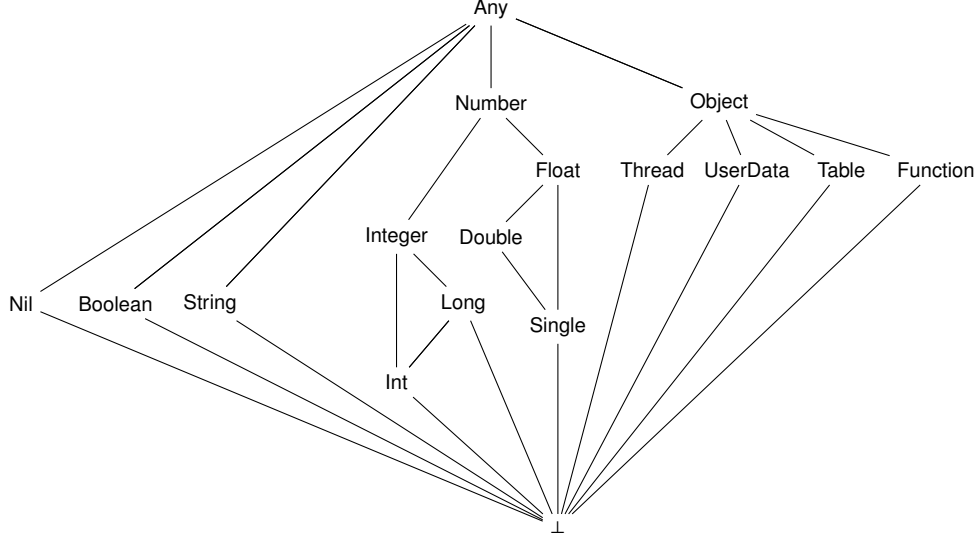


Figure 1: Hierarchy of Lua types.

We extend the $:$ notation to assign types not only to identifiers and expressions but also to type expressions admissible by σ_τ in programs. We use generic e and f for (possibly type annotated) expressions and r and s for (sequences of) statements (possibly containing expressions). Our type annotations are just ascriptions in the sense detailed in [17]. This means that type annotations do not lead to any type coercion or conversion in programs but must be respected whenever specified.

4.2. Type System Semantics

4.2.1. Type System Definitions

We formalize the semantics of Lua types by defining a logical system with the subtyping and type inference rules presented in Sections 4.2.2 and 4.2.3. Therein, we adopt typing contexts of the form $\Delta :: \Psi \cup \Xi \rightarrow \mathcal{L}(\tau)$, which are partial functions assigning type and program variables to type expressions. Whenever $x \in \text{dom } \Delta$ and $\Delta(x) = t$, we write interchangeably $\{x \mapsto t\} \in \Delta$ to denote a type assignment and $\Delta[x : t]$ to denote a type ascription in Δ . For two contexts Γ and Λ , we say that $\{x \mapsto t\} \in \Gamma \cup \Lambda$ iff $x \in (\text{dom } \Gamma \cup \text{dom } \Lambda)$ and, if $x \in \text{dom } \Gamma \cap \text{dom } \Lambda$, then $t = \cup(\Gamma(x), \Lambda(x))$, otherwise $t = \Gamma(x)$ or $t = \Lambda(x)$. Thus, $\Gamma \subseteq \Gamma'$ has the standard set-theoretic definition.

Given $\mathbf{f} \in C(\sigma_\tau)$, the set of free type variables in $\mathbf{f}$ is denoted by $FTV(\mathbf{f}) \subseteq \Xi$. Given a context Δ , we write $FTV(\Delta)$ to represent the set of free type variables in Δ . Moreover, given a code fragment $\mathbf{f} \in C(\sigma_\tau)$ and variables $\{x, x'\} \subseteq \Psi \cup \Xi$, we denote by $\mathbf{f}[x'/x]$ the code fragment obtained from $\mathbf{f}$ by substituting every free occurrence of x in $\mathbf{f}$ by x' . We take contexts instead of code fragments and type expressions instead of type variables to define extended notions of substitution.

We define type instantiations and generalizations using substitutions. Given $t \in \mathcal{L}(\tau)$, we denote by $t[x'/x]$ the substitution in t of each free occurrence of x by x' . Thus, we say that $t' = t[x'/x]$ is an instance of t . Generalizations are defined in the other way round.

<u>Constants :</u>	$\text{TBot} \frac{}{\Delta \vdash \perp \leq t} \quad \text{TCReFL} \frac{}{\Delta \vdash t \leq t \left[\begin{smallmatrix} \text{isTTag}(t), \\ t \neq \text{Any} \end{smallmatrix} \right]} \quad \text{TTOP} \frac{}{\Delta \vdash t \leq \text{Any}}$
<u>Variables :</u>	$\text{TVReFL} \frac{}{\Delta \vdash u \leq u' \left[\begin{smallmatrix} u = u' \vee \\ \Delta(u') = u \end{smallmatrix} \right]} \quad \text{TVar} \frac{\Delta \vdash t \leq t'}{\Delta \vdash u \leq t' \left[\begin{smallmatrix} \Delta(u) = t, \\ t' \neq u \end{smallmatrix} \right]} \quad \text{TForALL} \frac{\Delta \vdash t \leq t'}{\Delta \vdash \forall u. t \leq \forall u. t' \left[\text{ugFTV}(\Delta) \right]}$
<u>Numbers :</u>	$\text{TIL} \frac{}{\Delta \vdash \text{Int} \leq \text{Long}} \quad \text{TIINT} \frac{}{\Delta \vdash \text{Int} \leq \text{Number}} \quad \text{TLONG} \frac{}{\Delta \vdash \text{Long} \leq \text{Number}}$ $\text{TSINGLE} \frac{}{\Delta \vdash \text{Single} \leq \text{Number}} \quad \text{TDOUBLE} \frac{}{\Delta \vdash \text{Double} \leq \text{Number}}$ $\text{TSD} \frac{}{\Delta \vdash \text{Single} \leq \text{Double}}$
<u>Objects :</u>	$\text{TUSUP} \frac{\Delta \vdash t \leq t_i}{\Delta \vdash t \leq t_1 \cup \dots \cup t_n \left[\begin{smallmatrix} n > 1, \\ \exists i: 1 \leq i \leq n \end{smallmatrix} \right]} \quad \text{TUSUB} \frac{\Delta \vdash t_i \leq t}{\Delta \vdash t_1 \cup \dots \cup t_n \leq t \left[\begin{smallmatrix} n > 1, \\ \forall i: 1 \leq i \leq n \end{smallmatrix} \right]}$ $\text{TFUNC} \frac{\Delta \vdash t'_1 \leq t_1 \quad \Delta \vdash t'_2 \leq t'_2}{\Delta \vdash (t_1 \rightarrow t_2) \leq (t'_1 \rightarrow t'_2)} \quad \text{TSEQ} \frac{\Delta \vdash t \leq t' \quad \Delta \vdash t'' \leq t'''}{\Delta \vdash (t : t') \leq (t'' : t''')}$ $\text{TTAB} \frac{\Delta \cup \{t_i = t'_j\} \vdash t'_i \leq t''_j}{\Delta \vdash \{t_1 \mapsto t'_1, \dots, t_k \mapsto t'_k\} \leq \{t''_1 \mapsto t'_1, \dots, t''_n \mapsto t'_n\} \left[\begin{smallmatrix} k \geq 1, n \leq k, \\ \forall i, j: 1 \leq i, j \leq k \\ \{t'_1, \dots, t'_n\} \subseteq \{t_1, \dots, t_k\} \end{smallmatrix} \right]}$

Table 1: Subtyping rules.

4.2.2. Subtyping

We use the partial order symbol $\leq$ to denote the subtyping relationship between Lua types. Given types t and t' , we write $t' \leq t$, meaning that t' is a subtype of t . Whenever $t \leq t'$ and $t' \leq t$, we write $t = t'$. With the aid of our subtyping rules, type identity reduces to structural equivalence, thus supporting the usual practice of type identity comparison based on type shape.

We define inference rules for subtyping in terms of assertions of the form $\Delta \vdash t \leq t'$, meaning that context Δ entails t as a subtype of t' for each pair of type expressions $\{t, t'\} \subseteq \mathcal{L}(\tau)$. Due to the set-theoretic definition of contexts, assertions naturally inherit a weakening property: (WEAK) $\Delta \vdash t \leq t' \Rightarrow \Delta \cup \{x \mapsto t''\} \vdash t \leq t'$ for any $x \in \Psi \cup \Xi$ and $t'' \in \mathcal{L}(\tau)$. Inference rules providing a proof-theoretic definition of subtyping are presented in Table 1. In rule (TCReFL), we use an auxiliary meta-function *isTTag* to identify type expressions formed using just ground type tags.

Our subtyping rules resemble those in [12], which deal with the top element *Any* and basic, structured and universally quantified types. In particular, the (TFUNC) rule verifies subtyping relationships between functions by checking the contravariance of parameters and the covariance of return types. (TSEQ) defines sequence subtyping based on the lexicographic order subtyping

of the respective constituent types. Concerning tables, (TTAB) verifies subtyping relationships by ensuring that type equality of table keys leads to subtyping relationships between the respective field types. Moreover, our rules also deal with the bottom element ($\perp$), and numeric and union types. However, our rules are more syntax-driven than those in [12], in the sense that, given a subtyping relationship, at most one inference rule can be applied [17]. These subtyping rules define $\leq$ as a subtyping relation, thus supporting safe type substitution:

Theorem 4.1 (Subtyping relation). The binary relation $\leq$ is a subtyping relation.

Proof: We have to show that our inference rules define $\leq$ as a reflexive and transitive relation. We prove this by showing that rules (TREFL) $\Delta \vdash t \leq t$ and (TTRANS) $\Delta \cup \{t \leq t', t' \leq t''\} \vdash t \leq t''$ are derivable for every Δ and $\{t, t', t''\} \subseteq \mathcal{L}(\tau)$. We use structural induction in both cases.

Concerning (TREFL), the proof is developed by structural induction on $\mathcal{L}(\tau)$, the source of type expressions serving as the left and the right-hand sides of each subtyping relationship. The base cases are defined by basic and numeric types, type expressions formed just using the bottom and top type tags, other structured object type tags, variables, and the empty sequence and table. Functions, non-empty sequences and tables, unions and quantified type expressions define inductive cases. Each base case is verified in a single step by applying the respective subtyping rule. The inductive cases, characterized by a subtyping relationship between structured type expressions, are verified by applying object subtyping rules, which have their premises immediately checked since they correspond to inductive hypotheses.

Concerning (TTRANS), the proof is developed by structural induction on pairs of the form $(\mathcal{L}(\tau), \mathcal{L}(\tau))$, whose elements correspond to t and t'' in the inference rule conclusion, with an inner induction on t' . The proof is organized by cases based on the lattice structure presented in Fig. 1. Each base case is verified in a single step by applying the respective subtyping rule. The inductive cases are verified based on the induction hypothesis and on the transitivity of the $\leq$ relationship whenever it is applied to cases not containing structured types. The details are omitted here. ■

Now we show that the bottom-up application of our subtyping rules is deterministic (modulo the choice of fresh variable names):

Theorem 4.2 (Subtyping determinism). Lua subtyping, as defined by $\leq$, is deterministic.

Proof: Determinism is a consequence of the fact that only one subtyping rule can be applied to reduce the length of each well-formed $\leq$ subtyping expression. This is verified by structural induction on $\mathcal{L}(\tau)$ using our subtyping rules. Note that, whenever there is more than one rule with consequence matching a given subtype expression, such rules have mutually exclusive side-conditions. Moreover, subtyping expressions have finite length, and the application of each rule reduces the length of the focus relationship. Consequently, the subtyping verification procedure always eventually terminates. ■

4.2.3. Type Inference

Now we extend our notion of assertion to cover code fragments. We write assertions of the form $\Delta | \Delta' \vdash \mathfrak{f}$ to denote that a context Δ entails (the typing of) code fragment $\mathfrak{f} \in C(\sigma_\tau)$ and yields a new context Δ' for the inference of typing information in subsequent code fragments. We adopt this unusual syntax here to capture the lexical scoping of variables in Lua. Whenever $\Delta = \Delta'$, we safely omit one of the contexts from the assertion.

Table 2 presents the type inference rules for constants, variables and type expressions. In some of these rules, we use in side conditions auxiliary functions with the *is* prefix to identify value

<u>Constants :</u>	$\text{ESEQ} \frac{}{\Delta \vdash () : \cdot \langle \cdot \rangle} \quad \text{ETAB} \frac{}{\Delta \vdash \{\} : \cdot \text{Table}} \quad \text{NIL} \frac{}{\Delta \vdash \text{nil} : \cdot \text{Nil}}$
<u>Variables :</u>	$\text{SPEC} \frac{\Delta \vdash t \leq t'}{\Delta \vdash v : \cdot t'} \left[\begin{array}{l} t' \notin \Xi, \\ t = (v \in \text{dom } \Delta ? \Delta(v) : \text{Nil}) \end{array} \right] \quad \text{ABS} \frac{\Delta \vdash e : \cdot t}{\Delta \vdash e : \cdot \forall u . t} \left[\begin{array}{l} t \notin \Xi, \\ u \notin \text{FTV}(\Delta) \end{array} \right]$
<u>Basic Types :</u>	$\text{BOOL} \frac{}{\Delta \vdash c : \cdot \text{Boolean}} [isBool(c)] \quad \text{STR} \frac{}{\Delta \vdash s : \cdot \text{String}} [isStr(s)]$
	$\text{NUM} \frac{\Delta \vdash n : \cdot t}{\Delta \vdash n : \cdot u} \left[\begin{array}{l} isNum(n), \\ t = (u \in \text{dom } \Delta ? \Delta(u) : df(n)) \end{array} \right]$
<u>Numeric Types :</u>	$\text{INTEGER} \frac{}{\Delta \vdash n : \cdot \text{Integer}} \left[\begin{array}{l} isInteger(n), \\ isInt(n) \end{array} \right] \quad \text{LONG} \frac{}{\Delta \vdash n : \cdot \text{Long}} \left[\begin{array}{l} isInteger(n), \\ \neg isInt(n) \end{array} \right]$
	$\text{FLOAT} \frac{}{\Delta \vdash n : \cdot \text{Float}} \left[\begin{array}{l} isFloat(n), \\ isSingle(n) \end{array} \right] \quad \text{DOUBLE} \frac{}{\Delta \vdash n : \cdot \text{Double}} \left[\begin{array}{l} isFloat(n), \\ \neg isSingle(n) \end{array} \right]$
<u>Object Types :</u>	$\text{TAB} \frac{\Delta \vdash e_i : \cdot t_i \quad \Delta \vdash e'_i : \cdot t'_i}{\Delta \vdash \{e_1 \mapsto e'_1, \dots, e_n \mapsto e'_n\} : \cdot \{t_1 \mapsto t'_1, \dots, t_n \mapsto t'_n\}} \left[\begin{array}{l} n \geq 1, \\ \forall 1 \leq i \leq n : t_i \neq \text{Nil} \end{array} \right]$
	$\text{SEQ} \frac{\Delta \vdash e : \cdot t \quad \Delta \vdash e' : \cdot t'}{\Delta \vdash (e : e') : \cdot (t : t')} \quad \text{UNION} \frac{\Delta \vdash e : \cdot t'_1 \cup \dots \cup t'_k}{\Delta \vdash e : \cdot t_1 \cup \dots \cup t_n} \left[\begin{array}{l} n > 1, k < n \\ \forall 1 \leq i \leq k : t'_i \in \{t_1, \dots, t_n\} \end{array} \right]$

Table 2: Inference rules for constants, variables and type expressions.

types based on their lexical analysis. All these rules are relatively standard, except (SPEC), (NUM) and (UNION). The rule (NUM) assigns variables to be resolved later on in the type inference process to numeric values of unspecified type, taking into account previously defined type ascriptions or the default type for that value. (SPEC) captures the fact that each program variable has already appeared in the program, having its type determined by the respective context, or else the variable has not appeared yet and it is being (implicitly) attributed to the value `nil`, consequently having the Nil type. The (UNION) rule, concerning union types, is a bit peculiar but appears in [17].

For an expression e of type t , the corresponding type annotated code fragment $f \in C(\sigma)$ has the form $f = e : t$. Thus, we write $\Delta \vdash f$ or $\Delta \vdash e : t$, meaning that Δ entails the type t for e . For a statement s entailed by Δ , we write $\Delta \vdash s$. Although the inference of expression types is relatively standard in Lua, this is not the case for statements because they can refer to local and global program variables. Consequently, we assume that each typing context Δ is partitioned into a global Γ and a local Λ context, and also presume that weakening preserves context partitioning. Whenever context partitioning is not relevant, it can be safely omitted from assertions. We define inference rules for expressions and statements in Table 3.

In Tables 2 and 3, we only present inference rules for the imperative fragment of the Lua language since the object-oriented fragment is defined in [11] based on imperative constructions. These tables assume that we already know each predefined operator and function type, which is returned by the meta-function dt . We consider that dt always returns the most general type signature for each predefined operator and function, according to the language definition [11], and deal with coercion through subtyping. In addition, we assume therein that a numerical type defaulting function df has been provided, as detailed in Section 4.4.

The type inference rules in Tables 2 and 3 resemble those proposed for other imperative programming languages. For example, most inference rules regarding program expressions depend on subtyping entailments. Concerning conditional control structures, the respective rules have their conditions tagged with **Any** here because expressions of any type are allowed therein, even though just `nil` or `false` falsify their evaluation. Similar to other programming languages, rules (L_{ASGN}) and (G_{ASGN}) deal in analogous ways with local and global typing contexts in the respective assignments. However, our inference rules for control structures contain additional side conditions to establish restrictions for subsequent type inference steps. In addition, since Lua tables resemble heterogeneous arrays, our (C_{ASE}) rule is a bit peculiar since it has to join in a union all the possibly different resulting types obtained in indexed table accesses.

Note that rules (F_{UNCABS}) and (R_{ET}) do not have straightforward formulations because they are based on a coding scheme for recording and retrieving the return type of each function f in a respective ghost program variable out_f . (F_{ORGEN}) also adopts a coding scheme to connect the assertions related to generic `for` statements. Moreover, in many inference rules, we use the bar notation to refer to dereferencing and size adjustment procedures since sequences of types and values can be either completed, respectively with `Nil` and `nil`, or have some elements ignored, whenever these adjustments are required by program statements, such as in assignments and function calls/returns.

It is important to mention that the implementations of these rules (particularly those of R_{ET} and C_{ASE}) are more complex than their specifications in Tables 2 and 3, because Lua is polymorphic and does not follow a strict typing discipline. The language allows, for example, the assignment of a value to a program variable whose type is different from the previously assigned value type. Moreover, many `return` statements in the body of a function are admissible, potentially returning values of different types. So, in such situations, each respective inference rule has to collect all the resulting types in a single union type.

4.3. Type Inference Algorithm

In Section 4.2, we introduced a typing scheme covering almost the entire Lua language due to our interest in defining a type system that can be used in practice through a decision procedure. The original decision procedure for type inference in programming languages with parametric polymorphism was proposed in [18]. Here, we follow the same procedure: in order to infer the type of identifiers and expressions in a given program, we generate an abstract syntax tree from the given code and traverse it to determine type annotations for expression branches and leaves, as well as for identifiers. The typing of each leaf is determined by the inference rules in Table 2, whereas the rules in Table 3 guide the typing of each branch and control the effect of statements on expression and identifier types. To determine which rule to apply in a given situation, we use pattern matching. Note that the side conditions in some rules require the computation of logical and arithmetic operations, set inclusions, meta-function applications and type equality.

The type inference algorithm is used to deduce the most general admissible types for each expression and identifier in a program, even if no typing information is provided in the given

<u>Expressions :</u>	
PDFAPP	$\frac{\Delta \vdash t'_1 \leq t_1 \quad \Delta \vdash t_2 \leq t'_2 \quad \overline{\Delta \vdash e : t'_1}}{\Delta \vdash f(e) : t'_2} \left[\frac{dt(f)=}{t_1 \rightarrow t_2} \right]$
FAPP	$\frac{\Delta \vdash t'_1 \leq t_1 \quad \Delta \vdash t_2 \leq t'_2 \quad \Delta \vdash f : t_1 \rightarrow t_2 \quad \overline{\Delta \vdash e : t'_1}}{\Delta \vdash f(e) : t'_2} \left[\frac{f \notin}{\text{dom } dt} \right]$
UOP	$\frac{\Delta \vdash t'_1 \leq t_1 \quad \Delta \vdash t_2 \leq t'_2 \quad \Delta \vdash e : t'_1}{\Delta \vdash (o \ e) : t'_2} \left[\frac{dt(o)=}{t_1 \rightarrow t_2} \right]$
BOP	$\frac{\Delta \vdash \langle t'_a, t'_b \rangle \leq \langle t_a, t_b \rangle \quad \Delta \vdash t_2 \leq t'_2 \quad \Delta \vdash e : t'_a \quad \Delta \vdash f : t'_b}{\Delta \vdash (e \ o \ f) : t'_2} \left[\frac{dt(o)=}{\langle t_a, t_b \rangle \rightarrow t_2} \right]$
CASE	$\frac{\Delta \vdash e_1 : \{t_1 \mapsto t'_1, \dots, t_n \mapsto t'_n\} \quad \Delta \vdash e_2 : t}{\Delta \vdash (e_1 [e_2]) : (\cup \{t'_i t_i = t, 1 \leq i \leq n\}) \cup \{\text{Nil}\}} \quad \text{TYPEASCR} \frac{\Delta \vdash e : t}{\Delta \vdash (e : t) : t} \left[\frac{t=t' \vee}{(t \in \Sigma \wedge \Delta(t)=t')} \right]$
<u>Statements :</u>	
BLK	$\frac{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash s \quad \Gamma' \cup \Lambda' \Gamma'' \cup \Lambda'' \vdash r}{\Gamma \cup \Lambda \Gamma'' \cup \Lambda'' \vdash s ; r} \left[\frac{\Gamma \subseteq \Gamma', \Gamma' \subseteq \Gamma'', \Lambda \subseteq \Lambda', \Lambda' \subseteq \Lambda''}{\Gamma \subseteq \Gamma'', \Lambda \subseteq \Lambda''} \right]$
LBL	$\frac{}{\Delta \vdash 1} [isLabel(l)] \quad \text{GOTO} \frac{}{\Delta \vdash \text{goto } l} \left[\frac{l \in \text{dom } \Delta,}{\Delta \vdash \Delta(l)=x} \right]$
BRK	$\frac{}{\Delta \vdash \text{break}} \quad \text{REP} \frac{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash s \quad \Gamma' \cup \Lambda' \vdash e : \text{Any}}{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash \text{repeat } s \text{ until } e} \left[\frac{\Gamma \subseteq \Gamma', \Lambda \subseteq \Lambda'}{\Gamma \subseteq \Gamma', \Lambda \subseteq \Lambda'} \right]$
DO	$\frac{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash s}{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash \text{do } s \text{ end}} \left[\Gamma \subseteq \Gamma' \right] \quad \text{WHILE} \frac{\Gamma \cup \Lambda \vdash e : \text{Any} \quad \Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash s}{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash \text{while } e \text{ do } s \text{ end}} \left[\Gamma \subseteq \Gamma' \right]$
IF	$\frac{\Gamma \cup \Lambda \vdash e : \text{Any} \quad \Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash s \quad \Gamma \cup \Lambda \Gamma'' \cup \Lambda'' \vdash r}{\Gamma \cup \Lambda (\Gamma' \cup \Gamma'') \cup \Lambda' \vdash \text{if } e \text{ then } s \text{ else } r \text{ end}} \left[\frac{\Gamma \subseteq \Gamma', \Lambda \subseteq \Lambda'}{\Gamma \subseteq \Gamma', \Lambda \subseteq \Lambda'} \right]$
LASGN	$\frac{\Gamma \cup \Lambda \vdash e_j : t_j \quad \Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash (v_1, \dots, v_n) : \langle t_1, \dots, t_k \rangle}{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash \text{local } v_1, \dots, v_n = e_1, \dots, e_k} \left[\frac{1 \leq j \leq \min(n, k),}{\Lambda \subseteq \Lambda', (v_j : t_j) \in \Lambda'} \right]$
GASGN	$\frac{\Gamma \cup \Lambda \vdash e'_j : t'_j \quad \Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash e_i : t_i \quad \Gamma' \cup \Lambda' \Gamma'' \cup \Lambda'' \vdash \langle t_1, \dots, t_n \rangle : \langle t'_1, \dots, t'_k \rangle}{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash e_1, \dots, e_n = e'_1, \dots, e'_k} \left[\frac{1 \leq i \leq n,}{1 \leq j \leq k, \Gamma \subseteq \Gamma'} \right]$
RET	$\frac{\Gamma \cup \Lambda \vdash e : t}{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash \text{return } e} \left[(out_f : t) \in \Lambda \right]$
(L)FABS	$\frac{\Gamma \cup \Lambda \vdash t'_1 \leq t_1 \quad \Gamma \cup \Lambda \vdash t_2 \leq t'_2 \quad \Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash e : t'_1 \quad \Gamma' \cup \Lambda' \cup \{out_f : t'_2\} \Gamma'' \cup \Lambda'' \vdash s}{\Gamma \cup \Lambda \Gamma'' \cup \Lambda'' \vdash (\text{local function } f \text{ e s end})} \left[\frac{\Gamma \subseteq \Gamma', \Gamma' \subseteq \Gamma'', \Lambda \subseteq \Lambda',}{out_f \notin \text{dom } (\Gamma \cup \Lambda),} \right]$
FORNUM	$\frac{\Gamma \cup \Lambda \vdash e_i : t \quad \Gamma' \cup \Lambda' \cup \{v : t\} \Gamma'' \cup \Lambda'' \vdash s}{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash \text{for } v = e_1, e_2, e_3 \text{ do } s \text{ end}} \left[\frac{1 \leq i \leq 3,}{\Gamma \subseteq \Gamma'} \right]$
FORGEN	$\frac{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash \text{local } v_f, v_s, v_p = e_1, \dots, e_l \quad \Gamma \cup \Lambda' \Gamma'' \cup \Lambda'' \vdash v_f(v_s, v_p) : \langle t'_1, \dots, t'_k \rangle \quad \Gamma'' \cup \Lambda'' \Gamma''' \cup \Lambda''' \vdash s}{\Gamma \cup \Lambda \Gamma' \cup \Lambda' \vdash \text{for } v_1, \dots, v_k \text{ in } e_1, \dots, e_l \text{ do } s \text{ end}} \left[\frac{1 \leq i \leq k,}{\Gamma \subseteq \Gamma', \Lambda \subseteq \Lambda', (v_i : t'_i) \in \Lambda'',} \right]$
	$\left[\frac{v_f, v_s, v_p \notin \text{dom } (\Gamma \cup \Lambda)}{\Gamma \subseteq \Gamma', \Lambda \subseteq \Lambda'} \right]$

Table 3: Inference rules for expressions and statements.

source code. As usual, we presume that the given program is syntactically correct. We use a type reconstruction procedure to assign type schemes to program expressions and identifiers, keeping these associations in a global context through type variable bindings. Once the algorithm identifies an inference rule matching a program fragment, it proceeds bottom-up attempting to satisfy the rule premises. In this path upwards, the algorithm generates fresh type variables to serve as the initial type schemes assigned to identifiers and expressions. It also abstracts away quantified variables from the definition of built-in functions and operators that are used in the given source code, substituting them by fresh type variables. Later on, the algorithm proceeds downwards, performing unification.

Unification can bind type variables to more concrete type schemes as the algorithm proceeds. This is performed in an incremental way so as to solve the corresponding type equations, an alternative formulation of the type inference problem described in the literature [19]. The unification algorithm fails whenever it cannot find valid unifiers to proceed. In particular, the algorithm fails whenever it attempts to unify a type variable with a type expression wherein the type variable occurs [17]. On the other hand, whenever this procedure succeeds, the produced assignments determine the most general type admissible for each expression and identifier in the program. We can thus verify the following result (modulo the choice of fresh variable names):

Theorem 4.3 (Decidability of Type Inference). The type inference algorithm always finds the most general types admissible for all the expressions and identifiers in each given syntactically correct Lua program or else it fails, rendering the program untypable.

Proof: Termination is a direct consequence of the fact that only one type inference rule can be applied to each code fragment. It is verified by structural induction on $\mathcal{L}(\sigma_+)$ using the proposed type inference rules. Note that whenever there is more than one rule with consequence matching the given code fragment, these rules have mutually exclusive side conditions. Type inference rules also contain subtyping entailments, so termination depends on subtyping determinism. Since code fragments have finite length, and the application of each rule reduces the length of the focus fragment, the type inference procedure always eventually terminates. It either produces a set of assignments of type variables to type schemes for all the expressions and identifiers in the given program or a failure due to unification mismatch. The production of principal types is a direct consequence of the properties of unification. Details are omitted here. ■

4.4. Optimizing Code Transformation Scheme

Usually, programmers do not need to declare the type of each program variable in using dynamically interpreted languages. This programming freedom comes at the cost of hindering the usage of the full processing power available in multicore architectures. Such architectures cannot execute interpreted code natively, and runtime interpretation is often inefficient. Whenever typing information is inferred before execution, however, the runtime architecture can take advantage of the inferred types to exploit the available resources and deliver enhanced performance.

In our case, the types inferred by our algorithm may not be sufficient to ensure the generation of optimal code for the chosen architecture. This happens because numerical values and predefined functions are overloaded, and some contextual information generated by the typing algorithm is lost during the incremental computation of a solution for the corresponding type equations.

With the purpose of tuning the type inference procedure [20], according to the numerical processing power available in the target runtime architecture, we assume the existence of a numeric type defaulting function $df: \mathcal{L}(\tau_a) \rightarrow \mathcal{L}(\tau_a)$ such that $df(t) \leq t$ (an instance of the more general scheme adopted in Haskell [21]). The given defaulting function is called whenever the

type inference algorithm is about to leave a context in which a numerical value is still assigned to a type variable and consequently must be resolved in that context.

In order to ensure code generation with enhanced performance for the chosen runtime architecture, we rely not only on the provision of a numerical type defaulting function but also on the separate compilation of candidate functions. We adopt an opportunistic strategy to select, in the given source code, candidate functions to be compiled separately and identify therein opportunities of function call parallelization.

We adopt a specific criterion for selecting in the given source code candidate functions for parallel execution. We require that such functions admit only numerical data (after coercion) as their arguments and return values. Our type inference algorithm is used to detect the fulfillment of this requirement. We also inspect the given source code to detect opportunities of function call parallelization. In the source code, we attempt to identify higher-order functions which take numerical functions and numerically indexed arrays as their arguments. These are called scheduling functions here. We use our algorithm again to detect such functions in the source code. In the most general case, additional candidate function requirements should be adopted, such as not sharing data structures with the rest of the program. However, the verification of this kind of requirement is out of the scope of the present paper.

The Source-to-Source Translator component of our model identifies candidate functions in the original program and substitutes their invocations by calls to the corresponding compiled versions, which are interfaced with the main program. The identified candidate functions, together with their inferred types, are provided as inputs to the Compiler Provider. This component transforms functions with numerical signature into compiled code. In addition, it substitutes scheduling functions by their respective parallelizing versions. It is important to mention that these high-level source code transformations can be performed in more automated and optimizing ways, by relying on techniques such as to perform automatic vector data decomposition [22, 23], and tiling and vectorization [24]. However, these transformations are usually implemented using complex static data-flow analyses, which are also out of the scope of the present paper.

The tool we have implemented transforms sequential executions of a Lua program into instances of MapReduce processes [25]. The MapReduce framework applies a `map` function to each given array element, generating an intermediate set of key-value pairs, and then uses a `reduce` function to compute a resulting answer set. In our implementation, `map` has the standard behavior, but `reduce` is just an identity function. This framework implements the Device Manager component of our model, which fetches data concerning available resources from the underlying computing architecture, performs parallelization optimizations, and executes each task in parallel whenever possible. It determines the maximum number of processing threads, performs processor allocation, and executes task parallelization. Due to the Lua language built-in support to foreign functionality invocation, the respective computations are performed with minimal overhead.

Since Lua is not defined together with formally specified semantics (although a fragment is formally defined in [15]), we can only argue informally about the preservation of the Lua language semantics by this code transformation scheme. In fact, correctness is ensured solely due to the aforementioned language interfaceability and extensibility features.

4.5. Implementation Details

We have implemented our type inference and code transformation procedures in the form of an open-source Lua pre-compiler. The implementation of the Source-to-Source translator follows the approach proposed in [26] to treat compilation as an abstract syntax tree (AST) transformation

process. We use AntLR4 [27] to generate a parser for Lua programs potentially annotated with typing information. Once this parser is applied to a possibly annotated program, it produces a corresponding AST, which is later on traversed by the type inference algorithm. This process transforms the original AST into a type annotated one. Together with the chosen type defaulting function, the annotated AST serves as an input for a code transformation process.

Our tool relies on the data structures of AntLR4 [27] and the language implementation patterns described in [28]. In particular, we instantiate the pattern of symbol tables with nested scopes to deal with the lexical scoping scheme adopted in Lua. The type inference procedure is implemented as a two-phase process: the first phase collects all program symbols and generates type variables in a table while distinguishing between local and global symbols. The second phase performs static semantics verification procedures and type inference.

Our implementation of type inference follows the algorithm described in [19]. It is based on freshly generated type variables, so-called unknowns, as placeholders for type schemes assigned to type expressions. The set of type equations generated due to the AST traversal procedure is incrementally resolved by unification after static semantics verification is concluded with success.

It is possible to choose the level of strictness adopted in our type inference procedure. Although Lua follows a non-strict typing discipline, we find it convenient to tune the type inference strictness level for specific purposes, such as verifying compliance with specific coding patterns. The non-strict policy, which is compatible with the Lua standard implementation, leads the unification algorithm to recover any failure as soon as it occurs by creating a union of the type inferred so far with the one that gave rise to the failure. This happens with multiple `return` statements that produce values of different types and assignments that override program variables with values of distinct types. A super-strict policy disallows this kind of recovery. A strict policy just requires that any program variable be initialized before its first usage. The decision to allow strictness tuning using the command line of our pre-compiler leads the implementation of the proposed type inference rules to behave differently depending on the chosen strictness level.

We adopt `lua/luajit` and `g++` as the permitted instances of the Compiler Provider component of our architectural model due to their availability in many different platforms and because of their excellent performance. The chosen code generator is invoked after code transformation.

We have also implemented parallelizing versions of the Lua frontends in the form of bare-and-bone open-source tools [29]. They instantiate the Device Manager component of our reference architecture using a library of functions written in C++. We reuse Phoenix++ [30] in our prototype, the state-of-the-art multicore-based MapReduce framework. Our implementation exposes to the main program just the interface of the `map` function, which is hardcoded using the Lua C API.

5. Examples and Evaluation

5.1. Prime Factorization Example

This section uses the problem of determining the prime factorization of a given positive integer number as an example. Factorization is important, for example, in verifying the security of public-key cryptosystems, something closely connected to the difficulty of factoring the adopted numeric keys. Prime factorization is a computationally hard problem, for which no polynomial-time deterministic algorithm for classical computers is known at present. Consequently, parallel and distributed computing is a potential alternative for reducing processing times.

We present here a solution to the prime factorization problem based on the trial division algorithm. Given an integer n , we use the Sieve of Eratosthenes to generate all prime numbers

```

function pFacOfN(p,n)
  if (p <= 1 or n % p ~= 0) then
    return 0
  else
    return pFacOfN(p, n / p) + 1
  end
end

function sieve(n)
  local t = {}
  for i = 2,n do
    t[i] = 1
  end
  local limit = n^0.5
  for i = 2,limit do
    if (t[i]==1) then
      for j = i*i,n,i do
        t[j] = 0
      end
    end
  end
  local p,j = {},1
  for i = 2,n do
    if (t[i]==1) then
      p[j],j = i,j+1
    end
  end
  return p
end

function map (f, a)
  local b = {}
  for k,v in pairs(a) do
    b[k] = f(v)
  end
  return b
end

function filter (p)
  return pFacOfN(p,arg[1])
end

primes = sieve(arg[1])
factors = map(filter, primes)
result = "The prime factorization of "..
  arg[1].." is "
if (tonumber(arg[1]) == 1) then
  result = result.."1"
else
  for k,v in pairs(factors) do
    if factors[k] ~= 0 then
      if content then
        result = result.."*"
      else
        first = true
      end
      result = result..primes[k]..
        "~"..factors[k]
    end
  end
end
print(result)

```

Figure 2: Prime factorization program in Lua (sequential version).

up to n and then determine which of these primes divides n , thereby being factors of n . This procedure can be performed sequentially or in parallel, exploiting the maximum number of concurrent threads k admissible by the underlying computing architecture. A concurrent solution can be formulated considering that computations happen in rounds so that, in each round, the primality of k numbers is tested in parallel.

Our implementation of the trial division algorithm is presented in Fig. 2. This implementation consists of the functions `pFacOfN`, `map`, `sieve` and `filter`, and the main program. The function `pFacOfN` computes the number of times a prime number p divides the given integer n . The computed result corresponds to the exponent of p in the factorization of n . If the prime p does not divide the given integer n , then the computed exponent is zero. Otherwise, the function is applied recursively to n/p , and one is added to the value to be returned. Considering that n is obtained from the command line, function `filter` just applies `pFacOfN` to p and n . Regarding the function `map`, once it is applied to a given function and a table, it returns a new table containing the application of the function to each element of the table. In the main program, `map` is applied to `filter` and to the table returned by function `sieve`, which computes all the primes up to n .

The main program in Fig. 2 coordinates the execution of all these functions. The program takes the number to be factored from the command line, performs the required computations and sends the resulting factorization to the standard output.

5.2. Type Inference of Prime Factorization

Now we illustrate the application of our type inference algorithm in determining the types of some functions in Fig. 2. We present here just simplified type inference derivations, wherein (WEAK)

applications and subtyping verifications are omitted. Our choice of numerical type defaulting falls on the standard function, which assigns `Number` to any numerical value of unspecified type.

We begin with the inference of the type of `pFactOfN`. The derivation in Fig. 3 illustrates the behavior of our type inference procedure when applied to recursive functions without side effects. The inference procedure relies, among others, on the type of the predefined operations `+`, `%` and `/`, which take two arguments of type $\cup\langle\text{String}, \text{Number}\rangle$, perform coersions of `String` values to numbers and return values of type `Number`.

Since the rule (FABS) application in (1), we know that `pFactOfN` has schematic type $v_1 \rightarrow v_2$. Due to the applications of (SEQ) in (2) and (4), we also know that v_1 is bound to the sequence type $\langle v_3, v_5 \rangle$. Moreover, the application of (BOP) in (13) attempts to unify v_3 , as well as v_5 , with $\cup\langle\text{String}, \text{Number}\rangle$. However, the first attempt fails since v_3 has already been bound to `Number` due to the application of (BOP), (SPEC) and (NUM) in (9), (10) and (11) respectively. In particular, the unification fails because `Number` is a subtype of $\cup\langle\text{String}, \text{Number}\rangle$. Finally, the application of (NUM) and (RET) respectively in (29) and (30) yields the unification of v_2 with `Number`. So, we have a complete formal derivation that `pFactOfN` has the following type:

`pFactOfN:.(Number,U (String, Number)) -> Number`

Another interesting example consists in the inference of the type of `map`. The type derivation in Fig. 4 illustrates the behavior of our type inference procedure once applied to higher-order imperative functions. In the present case, type inference relies on the type of the predefined function `pair`, which takes a table as an argument and produces, as a result, an iterator function, the original table and the value `nil`. The returned iterator function, whenever applied to the original table and one of its indexes, produces the respective (index,value) pair, or else it returns `nil`, when no additional pair remains to be returned.

The derivation of the `map` type is more complex than that of `pFactOfN` because of its imperative character. In particular, notice in Fig. 4 that an order is enforced in the evaluation of local assertions by the side conditions of the applied inference rules. In (7), say, the assignment must be subject to type inference before the `for` loop, because the reverse would not satisfy the requirement of inclusion of the local context of the former assertion into that of the latter assertion.

Similarly to the `pFactOfN` function, we know since (1) that `map` has schematic type $v_1 \rightarrow v_2$ and that v_1 is bound to $\langle v_3, v_4 \rangle$, due to the sequence of (SEQ) applications in (2) and (4). In the body of the function, because of (8) and (9), we also know that the type of `b` is `Table`, due to the unification of v_6 with `Table` following (LASGN). Our procedure unfolds the inference of the types in the `for` loop into the inference of the types in a respective local assignment (13), function call (23) and global assignment in the loop body (30) involving ghost variables. The type inference of this local assignment depends on the type of `pairs`, which receives a table as an argument ($\{v_9 \mapsto v_{10}\}$, schematically) and returns an iterator function, the original table and the `nil` value (which have respective schematical types $\langle\{v_9 \mapsto v_{10}\}, v_9\rangle \rightarrow \langle v_9, v_{10}\rangle, \{v_9 \mapsto v_{10}\}$ and `Nil`). The type inference of the loop statement `b[i]=f(v)` captures the intuition that the type of each resulting table element is either `Nil` (due to the case in which the index does not correspond to any table element) or that resulting from the function application.

By choosing freshly generated type variables as they are required in the inference process and following the unifications in this process, we determine that `map` has the following type:

`map:.( _v10 -> U ( _v15, Nil), { _v9 | -> _v10 }) -> Table`



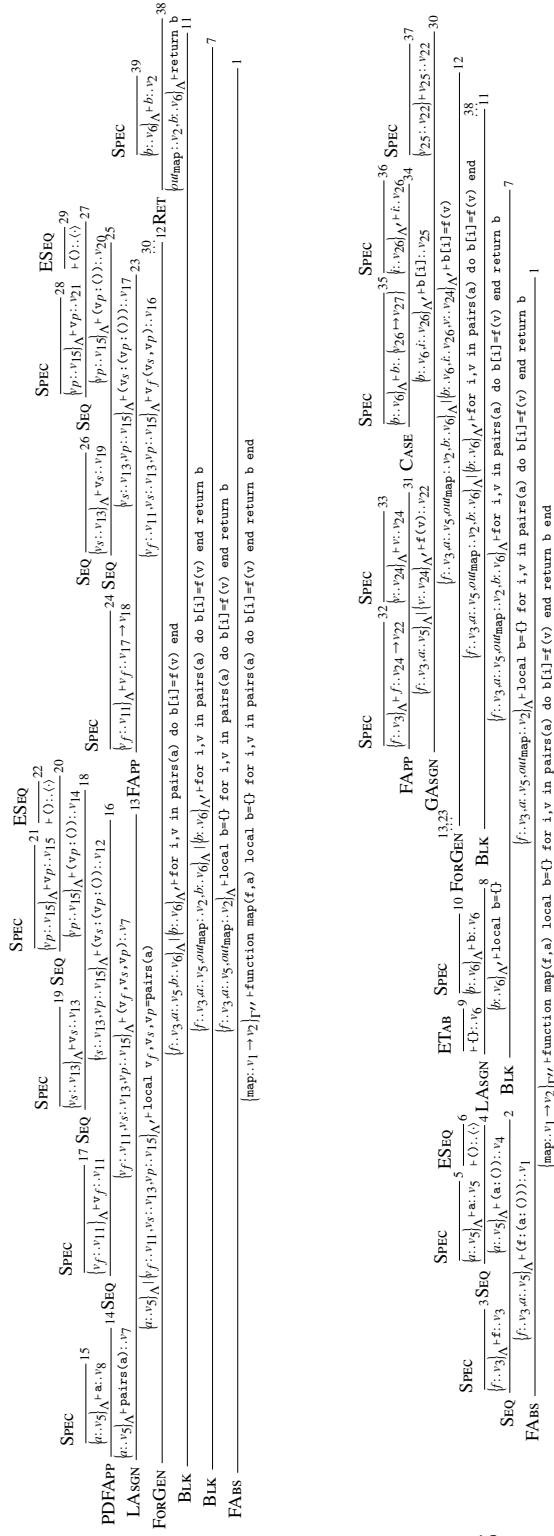


Figure 4: Simplified type inference of map.

Type variable assignment (Γ) and binding (Δ): (2): $v_1 \mapsto (v_3 : v_4) \Delta$;
 (does not include freshly generated variables (4): $v_4 \mapsto (v_5 : \langle \rangle) \Delta$;
 (9): $v_6 \mapsto \text{Table } \Delta$;
 (12): $v_6 \mapsto \langle v_9, v_{10} \rangle, v_{26} \mapsto v_{10} \mapsto v_9 \Delta$;
 (14): $\text{add}(\text{pairs}) = \{v_9 \mapsto v_{10} \mapsto \langle \langle v_9 \mapsto v_{10} \rangle, v_9 \rangle \mapsto \langle \langle v_9 \mapsto v_{10} \rangle, v_9 \rangle \mapsto \langle \langle v_9 \mapsto v_{10} \rangle, v_9 \rangle \mapsto \text{Nil} \Delta$;
 (15): $v_8 \mapsto v_5 \Delta$;
 (16): $v_7 \mapsto \langle v_{11}, v_{12} \rangle, v_{11} \mapsto \langle v_9 \mapsto v_{10} \rangle, v_9 \mapsto \langle v_9, v_{10} \rangle, v_{13} \mapsto \langle v_9 \mapsto v_{10} \rangle, v_{15} \mapsto \text{Nil} \Delta$;
 (18): $v_{12} \mapsto (v_{13} : v_{14}) \Delta$;
 (20): $v_{14} \mapsto (v_{15} : \langle \rangle) \Delta$;
 (23): $v_{18} \mapsto v_{16} \Delta$;
 (24): $v_{17} \mapsto \langle \langle v_9 \mapsto v_{10} \rangle, v_9 \rangle, v_{18} \mapsto \langle v_9, v_{10} \rangle \Delta$;
 (25): $v_{17} \mapsto \langle v_{19} : v_{20} \rangle \Delta$;
 (26): $v_{19} \mapsto v_{13} \Delta$;
 (27): $v_{20} \mapsto \langle v_{21} : \langle \rangle \rangle \Delta$;
 (28): $v_{21} \mapsto v_{15} \Delta$;
 (32): $v_{13} \mapsto \langle v_{24} \mapsto v_{22} \rangle \Delta$;
 (34): $v_{25} \mapsto \langle v_{27}, \text{Nil} \rangle \Delta$;
 (37): $v_{25} \mapsto v_{22} \Delta$;
 (39): $v_2 \mapsto v_6 \Delta$

It should be noted that the table type resulting from the `map` function application is generic rather than specific because Lua requires that, to avoid runtime errors, variables containing data structures must be initialized before indexed accesses take place. This is enforced by the assignment `local b = {}` in the body of function `map`. If this requirement were not present in the language specification, the inferred return type would have been $\{v_9 \mapsto v_{15}\}$ instead of `Table`.

It is important to mention that the pre-compiler we have implemented is capable of inferring the types not only of `pFacOfN`, `map`, `sieve` and expressions in the main program of Fig. 2, but also of any syntactically correct program, including those with non-strict assignments or which implement typable high-order combinators, such as `F` (presented in [31]) and `S`.

5.3. Concurrent Implementation and Execution of Prime Factorization

Concerning the automated transformation of the sequential prime factorization program into its respective concurrent implementation, first note that `pFacOfN` and `filter` satisfy the posed requirements to be regarded as candidate functions for parallelization. Moreover, `map` fulfills the requirements to be considered a scheduling function. The Source-to-Source translator component of our reference architecture takes the original source code file and transforms it into another one wherein these functions are so identified through comments in the code.

In the next step, the Compiler Provider component generates compiled code for each candidate function. If the target language is Lua, the generated bytecode is embedded into the transformed source code. Otherwise, this component generates C code that is then compiled in a dynamic library (with the same filename as the original source file) to be subsequently linked during the transformed code execution.

The concurrent execution of the program is initiated by invoking some modified version of the Lua frontend (the Lua interpreter or the LuaJIT compiler). The chosen tool takes the transformed program filename and the number to be factored from the command line. Then it reads the transformed source code, and executes the main program in a Lua state wherein the original `map` function is substituted by a hardcoded version implementing a MapReduce process. In the present case, for each prime factor, the `filter` function is activated in parallel batches, depending on the number of processors available in the underlying architecture. A new Lua state is created for each `filter` call if the generated code consists in bytecode. Otherwise, the compiled version of this function is dynamically loaded and executed. The resulting exponent set is used by the main program in the presentation of the prime factorization of the given number.

5.4. Performance Evaluation

We conducted some experiments to validate the idea that typing information can be used to enhance the performance of Lua programs when they are executed in multicore architectures. In particular, we adopted the example program in Fig. 2 for benchmarking purposes. Considering that the execution time of prime factorization depends on the bit-length of each number to be factored, we chose integer numbers in the range from 200000 to 40000000 as inputs since integers in this range produce distinguishable and measurable execution time statistics. We randomly selected 100 numbers in this range to form data samples. For each integer input chosen, we took the arithmetic mean of the total (user plus system) execution time of 10 runs of our factorization program in order to dilute any warm-up time required to load and keep in memory data structures that are recurrently needed for program execution.

Our statistics were collected just on single CPU architectures. There are significant differences between these statistics depending on the CPU model, microarchitecture, clock speed, number of

Table 4: Main Characteristics of each Benchmarking Architecture.

Single CPU Computer	atom x5-z8350	core i7-6500U	epyc 7R32*
Architecture	x86-64	x86-64	x86-64
Clock Speed (in GHz)	1.4	2.5	1.8
# Cores per CPU	4	2	8
# Threads per Core	4	4	2
L1d Cache (in KB)	24	32	32
L1i Cache (in KB)	32	32	32
L2 Cache (in KB)	1024	256	512
L3 Cache (in MB)	2	4	16
RAM (in MB)	2048	8192	16384
Native Linux OS	Fedora 31	Fedora 30	Red Hat Enterprise 8.3
Linux Kernel	5.5.6	5.6.13	4.18.0
User Interface	XFCE	KDE	XTerm

* Restricted setting chosen using the AWS EC2 environment.

cores per CPU, number of threads per code, available memory and other runtime features. Due to this reason, we conducted our benchmark adopting three different architectures, which are detailed in Table 4. We run the example program on each selected architecture with each candidate function substituted by the respective dynamically loadable C compiled version. The example program was executed on each architecture for each integer in the respective input sample using the Lua 5.3 interpreter [11], the LuaJIT 2.05 compiler [32] and modified versions of these tools that implement scheduling functions according to our parallelizing transformation scheme.

Fig. 5 shows the observed execution times for each architecture. In each box, we present a comparison of the execution times of the prime factorization program using `lua` (presented in red), `lua_jit` (in yellow) and the modified versions of these tools (respectively in blue and green). Each individual box provides evidence that the execution times obtained using the Lua interpreter are speed up by our transformations (since the red curves invariably appear above the blue ones). Still, the same does not necessarily happen regarding LuaJIT. These boxes also show that some transformation overhead is noticeable in the computations related to inputs of small bit-length. Comparing the boxes in a column to each other, we also notice that computation time is positively affected by the addition of memory (RAM and cache) and computing power (clock speed, number of CPU cores and threads per core).

On the left column of the figure, we present data concerning the execution times of the original program, whereas, on the right, we show execution time statistics for a heavily load version of our program, developed for comparison purposes. The left column shows that, by running the prime factorization program using the modified Lua interpreter, the obtained execution times for numbers of large bit-length are shorter than those observed in the execution using the original Lua interpreter. However, on average, the overall speedup is not statistically significant (as signaled in the figure through $\bar{\Delta} \cong 0$). Moreover, this slightly enhanced performance is not repeated in the case of LuaJIT, not only due to the exceptional performance of the original tool, but also because of the aforementioned transformation overhead. Analyzing this situation in more detail, we notice that the overall execution time of our prime factorization program is dominated by

function `sieve`. Indeed, more than 85% of the execution time of the program is spent running this function, according to the LuaJIT profiler (available in LuaJIT 2.1).

In order to clarify the performance improvements yield by our transformations, we produced a slightly modified heavy load version of our prime factorization program, which also computes $100!$ recursively each time function `pFactOfN` is called. In this case, at most 45% of the execution time is spent running `sieve`. So, there is a detachment of the curves representing the original program execution times from those corresponding to the execution of the respective transformed versions. In this case, both the modified `lua` and `luajit` tools present better performance than non-modified versions. Moreover, according to the non-parametric Wilcoxon signed-rank test [33], the speedup is statistically significant (in the figure, this is represented through $\bar{\Delta} > 0$). The average overall speedup obtained using the modified `lua` interpreter is always greater than 190%, regardless of the adopted CPU. In the case of LuaJIT, the average speedup is not smaller than 15%. The column on the right of the figure illustrates these improvements.

Our benchmarking experiment provides evidence that the proposed parallelizing transformations do not deliver significant performance improvements when the execution of the functions to be parallelized corresponds to a small portion of the original total execution time, as expected and predicted by Amdahl's Law [34]. But, when the parallelized functions are significantly more computationally intensive than the rest of the program, performance improvements are noticeable and speedup significant. Moreover, these results are scalable because that they improve with adding more memory and computing power to the adopted runtime architecture.

6. Related Work

The research reported in this paper is a result of the collaboration between the authors based on their previous experience on programming language semantics and compiler construction [26, 35], as well as on the definition and implementation of infrastructures for supporting parallel and distributed computing [22, 7, 8].

We have proposed an optional type system for the Lua language that integrates union types, parametric polymorphism and subtyping. This type system is defined in terms of typing and subtyping rules, which formally support the implementation of a type inference algorithm. Our work builds upon historical results on type systems for applicative languages [18, 16, 12, 17], which are neither of optional usage nor aim at imperative dynamic languages. Although Lua supports an object-oriented programming flavor, it is not an object-oriented language, and so it is compatible with standard unification and type inference algorithms. Nevertheless, recent research on dependent types for object-oriented languages recognizes the difficulty of integrating type systems with subtyping into real programming languages [36]. A more systematic comparison of our work with other optional typing schemes for dynamic languages is presented in Table 5.

Many other optional typing schemes have been proposed for dynamic languages: [1] defines a type system for Scheme, based on recursive and union types, polymorphism and subtyping; [2] proposes a type system with subtyping for a subset of Python, covering polymorphic functions, class inheritance, exceptions and modules; and [5] describes a type system that relies on intersection and union types, structured tuple and object types, as well as parametric polymorphism in Ruby. The first of these type systems has considerably influenced our research. There are also some typing scheme proposals for Javascript. For example, Typescript — an explicit typing specification language for Javascript, which additionally admits modules, classes and inheritance — has a type system based on recursive types that accepts programs with dynamic type errors, thus

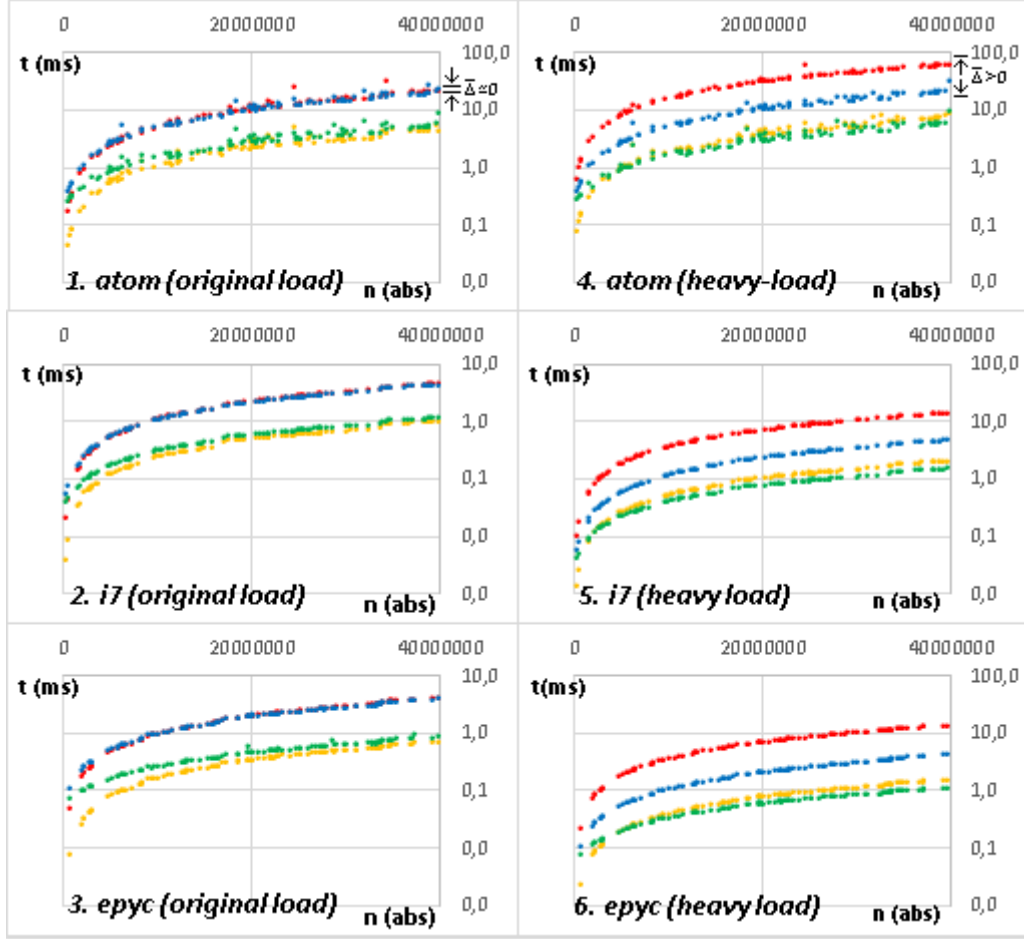


Figure 5: Prime factorization execution time plots for 100 randomly generated integer input samples, using lua original and modified versions (respectively drawn in red and blue) and luajit original and modified versions (respectively drawn in yellow and green).

being unsound (Featherweith Typescript) [4] while a different type system focuses on providing safety and performance guarantees and therefore rejects such programs (Safe Typescript) [37].

Concerning the Lua language, two typing schemes appear in the literature: Typed Lua [9] and Pallene [10]. Typed Lua is defined with a type-checking tool that covers the original language with extensions to increase abstraction and modularity. In contrast, Pallene is an optional static type companion to Lua that produces runtime checks to ensure dynamic type safety. We have preferred to stick to (a fragment of) the original Lua definition while developing a type inference tool that is independent of the Lua runtime environment since this seems to be a convenient configuration for exploiting computing resources available in parallel and distributed architectures.

We have been concerned with the effective execution of Lua programs in parallel and distributed architectures. Regarding this, it should be noted that both the Lua standard virtual machine

Table 5: Dynamically Typed Programming Languages with Their Optional Typing Scheme Proposals.

Language Name	Year of Original Definition	Supported Paradigm(s)	Original Typing Scheme	Optional Scheme Proposal	Typing	Authors (Year of Publication)	Ref.
Scheme	1975	functional, imperative	untyped	<i>Typed Scheme</i> : Static type inference procedure	Static type inference procedure	Tobin-Hoschsadt and Fellesen (2008)	[1]
Pyphon	1992	imperative, functional, object-oriented	dynamically-typed	<i>RPython</i> : Static type inference procedure	Static type inference procedure	Maia, Moreira and Reis (2011)	[2]
Lua	1993	imperative, functional, object-oriented	weakly dynamically-typed	<i>Typed Lua</i> : Static type verification procedure <i>Palene</i> : Mixed static and dynamic type verification procedure	Static type verification procedure Mixed static and dynamic type verification procedure	Maidl, Carenhas and Ierusalimsky (2014) Gualandi and Ierusalimsky (2018)	Mas- [9] and [10]
Ruby	1995	imperative, object-oriented	dynamically-typed	<i>Diamondback</i> : mixed static type inference and dynamic type verification based on constraint resolution	<i>Ruby</i> : Static type inference and dynamic type verification procedure based on constraint resolution	Furr, An, Foster and Hicks (2009)	[5]
Javascript	1995	imperative, object-oriented	untyped	<i>Featherweight script</i> : Static type inference procedure <i>Safe Typescript</i> : mixed static type inference and dynamic type verification procedure	<i>Type</i> : Static type inference procedure mixed static type inference and dynamic type verification procedure	Bierman, Abadi and Torgersen (2014) Rastogi and Others (2015)	[4] [37]

[11] and the state-of-the-art just-in-time compiler [32] execute generated bytecode presuming the existence of a single thread of control. Consequently, they do not provide built-in facilities for the transparent usage of multiple cores available in parallel and distributed architectures. A specific project aims to provide Lua with numerous parallel processing capabilities, called Torch [38], but focuses on machine learning algorithms. Another approach that is complementary to our research is to implement different concurrency patterns directly in Lua, as proposed in [39].

We showed that typing information is useful to enhance the performance of Lua programs when they are executed in multicore architectures. Regarding parallel and distributed architectures in general, alternative typing schemes have been proposed with this goal in mind. A feasible approach is to adopt a specific optional typing scheme to deal directly with the requirements of parallel and distributed architectures, as proposed in [40], but this brings the complexity of such architectures to the core of the type system definition. An alternative approach is to develop a parallel/distributed programming language with a built-in optional type system from scratch. This

path has been pursued in the development of the Julia language for scientific computing [41]. It supports union, recursive, parametric higher-order types and has a multiplatform implementation based on the LLVM compiler infrastructure [42].

There are many opportunities for extension and improvement of our work. So far, we have focused on improving the execution of Lua programs in multicore architectures. The extension of this work towards distributed computing does not seem challenging since one can adopt strategies like replicating runtime components [35] or entire virtual machines [22] at different computing locations. Regarding architectures with multiple GPUs and VPUs, a recent proposal that extends Phoenix++ in this direction appears in [43]. Moreover, the LLVM infrastructure can also be adopted as a more manageable and optimizing runtime infrastructure. However, the use of entirely automatic parallelization strategies, as outlined in [44], does not seem trivial in Lua, since detecting and managing side effects in imperative programs is a hard problem. Nevertheless, the related literature is a rich source of ideas, such as the usage of vector data decomposition [22, 23], and automatic tiling and vectorization [24] procedures.

7. Concluding Remarks

The research described in this paper corresponds to a substantial programming language engineering effort aiming to develop tools for efficiently executing Lua programs in parallel and distributed architectures. The main challenge we had to deal with in this effort was to integrate theoretical developments concerning type systems for programming languages with conceptual/practical ideas on type inference and code generation for dynamic languages, apart from dealing with real-world multicore architectures.

We have defined a type system that integrates union types, parametric polymorphism and subtyping; a static type inference algorithm; and a code transformation scheme suitable for enhancing the execution of Lua programs in multicore architectures. Subtyping is deterministic, and type inference is decidable in this system. We have also provided empirical evidence that this scheme brings statistically significant performance improvements to the execution of Lua programs in these architectures.

The neat definition of Lua facilitates the formulation of subtyping and type inference procedures. Unsafe constructs are not allowed by the language, such as type downcasts, pointer dereferencing and explicit memory handling. Complex type constructs, such as object-oriented and universal recursive types, can only be simulated using syntax sugaring, defined in terms of built-in data types. In addition, Lua execution environments have small memory footprints that ensure respectable performance baselines. These characteristics made possible the development of decidable and compositional subtyping and type inference procedures, as well as an optimizing code transformation scheme, for executing Lua programs in parallel and distributed architectures.

Our research so far points to promising directions for future research. We plan to provide enhanced automation for code generation and parallelization. Moreover, we would like to investigate how strictness tuning in particular application domains and target architectures can be exploited. In addition, it may be worthwhile studying the transportation to our theoretical and practical developments to other dynamic programming languages and computer architectures, particularly those adopted in machine learning, big data analytics and statistics [44].

References

- [1] S. Tobin-Hoschardt, M. Fellesen, The Design and Implementation of Typed Scheme, in: Proc. 35th Int. ACM Symposium on Principles of Programming Languages (POPL 2008). San Francisco, USA, ACM, 395–406, 2008.
- [2] E. Maia, N. Moreira, R. Reis, A Static Type Inference for Python, in: Proc. 5th Int. Workshop on Dynamic Languages and Applications (DyLa 2011). Zurich, Switzerland, 2011.
- [3] R. Ierusalimsky, L. H. de Figueiredo, W. C. Filho, Lua — An Extensible Extension Language, *Software, Practice and Experience* 26 (6) (1996) 635–652.
- [4] G. Bierman, M. Abadi, M. Torgersen, Understanding TypeScript, in: R. Jones (Ed.), Proc. 28th European Conference on Object-Oriented Programming (ECOOP 2014). Uppsala, Sweden, vol. 8586 of *Lecture Notes in Computer Science*, Springer, 257–281, 2014.
- [5] M. Furr, J. D. An, J. S. Foster, M. Hicks, Static Type Inference for Ruby, in: Proc. 24th Int. ACM Symposium on Applied Computing (SAC 2009), ACM, Honolulu, Hawaii, 1859–1866, 2009.
- [6] A. M. Maidl, F. Mascarenhas, R. Ierusalimsky, Typed Lua: An Optional Type System for Lua, in: Proc. 8th Int. Workshop on Dynamic Languages and Applications (DyLa 2014), ACM, Edinburgh, United Kingdom, 3:1–3:10, 2014.
- [7] R. Ribeiro, P. R. da Motta Júnior, Towards a GPU Abstraction for Lua, in: 2016 Int. Symposium on Computer Architecture and High-Performance Computing Workshops, SBAC-PAD Workshops, IEEE, Los Angeles, CA, USA, 13–18, 2016.
- [8] P. R. da Motta Junior, Declaring Lua data types for GPU code generation, in: Proc. 4th Int. ACM SIGPLAN Workshop on Software Engineering for Parallel Systems (SEPS 2017), ACM, Vancouver, Canada, 44–45, 2017.
- [9] A. M. Maidl, Typed Lua: An Optional Type System for Lua, PhD thesis, Department of Informatics, Pontifical Catholic University of Rio de Janeiro, 2015.
- [10] H. M. Gualandi, R. Ierusalimsky, Pallene: A statically-typed companion language for Lua, in: Proc. 22nd Brazilian Symposium on Programming Languages (SBLP 2018). São Carlos, Brazil, ACM, 19–26, 2018.
- [11] R. Ierusalimsky, L. H. de Figueiredo, W. Celes, Lua 5.3 Reference Manual [Online], Tech. Rep., Department of Informatics, Pontifical Catholic University of Rio de Janeiro, available: <https://www.lua.org/manual/5.3/manual.html>, 2017.
- [12] L. Cardelli, P. Wegner, On Understanding Types, Data Abstraction, and Polymorphism, *Computing Surveys* 17 (4) (1985) 471–523.
- [13] A. Goldberg, D. Robson, *Smalltalk-80: The language and its implementation*, Addison-Wesley, Boston, MA, USA, 608 pages, 1983.
- [14] S. P. Jones (Ed.), *Haskell 98 Language and Libraries – The Revised Report*, Cambridge University Press, Cambridge, England, 2003.
- [15] A. L. de Moura, R. Ierusalimsky, Revisiting coroutines, *ACM Transactions on Programming Languages and Systems* 31 (2) (2009) 1–31.
- [16] L. Damas, R. Milner, Principal type-schemes for functional programs, in: Proc. 9th Int. ACM Symposium on Principles of Programming Languages (POPL 1982). Albuquerque, New Mexico, USA, ACM, 207–212, 1982.
- [17] B. C. Pierce, *Types and Programming Languages*, MIT Press, Cambridge, MA, USA, 645 pages, 2002.
- [18] R. Milner, A Theory of Type Polymorphism in Programming, *Journal of Computer and Systems Science* 17 (1978) 348–375.
- [19] P. Hancock, A Type-Checker, in: S. L. Peyton-Jones (Ed.), *The Implementation of Functional Programming Languages*, Prentice Hall, Hertfordshire, UK, 163–184, 1987.
- [20] W. Dietl, M. D. Ernst, P. Müller, Tunable Static Inference for Generic Universe Types, in: Proc. 25th European Conference on Object-Oriented Programming (ECOOP 2011). Lancaster, UK, vol. 6813 of *Lecture Notes in Computer Science*, Springer, 333–357, 2011.
- [21] P. Hudak, J. Peterson, J. Fasel, A Gentle Introduction to Haskell (Version 98) [Online], available: <https://www.haskell.org/tutorial/index.html>, 1998.
- [22] P. R. da Motta Junior, An Abstraction for Parallel Programming, PhD thesis, Department of Informatics, Pontifical Catholic University of Rio de Janeiro, Brazil, in Portuguese, 2011.
- [23] J. C. et alli, Automatic Parallelization of Kernels in Shared-Memory Multi-GPU Nodes Share, in: Proc. 29th Int. ACM Conf. Supercomputing (ICS 2015). Newport Beach, California, USA, ACM, 3–13, 2015.
- [24] M. M. Pereira, R. C. F. Sousa, G. Araujo, Compiling and Optimizing OpenMP 4.X Programs to OpenCL and SPIR, in: B. R. de Supinski, et al. (Eds.), Proc. 13th Int. Workshop on OpenMP (IWOMP 2017), vol. 10468 of *Lecture Notes in Computer Science*, Springer, Stony Brook, NY, USA, 48–61, 2017.
- [25] J. Dean, S. Ghemawat, MapReduce: Simplified Data Processing on Large Clusters, *Communications of the ACM* 51 (1) (2008) 107–113.
- [26] C. H. C. Duarte, R. Ierusalimsky, On the Systematic Development of Compilers: A Case Study, in: P. Fritzson (Ed.), Proc. Poster Session of (CC’96) — International Conference on Compiler Construction, no. LITH-IDA-R-96-12

- in Technical Report Series of Department of Computer and Information Science, Linköping University, Sweden, 49–56, 1996.
- [27] T. Parr, *The Definitive ANTLR 4 Reference*, The Pragmatic Bookshelf, Raleigh, NC, USA, 328 pages, 2013.
 - [28] T. Parr, *Language Implementation Patterns*, The Pragmatic Bookshelf, Raleigh, NC, USA, 383 pages, 2010.
 - [29] R. Ierusalimschy, *Programming in Lua*, Lua.org, 4th edn., 2016.
 - [30] J. Talbot, R. M. Yoo, C. Kozyrakis, Phoenix++: Modular MapReduce for Shared-Memory Systems, in: *Proc. 2nd International Workshop on MapReduce and its Applications*, 9–16, 2011.
 - [31] R. Ierusalimschy, First-class functions in an imperative world, *Journal of Universal Computer Science* 23 (1) (2017) 112–126.
 - [32] M. Pall, LuaJIT, a Just-In-Time Compiler for Lua, available: <http://lua-jit.org/lua-jit.html>, 2005.
 - [33] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 6 (1) (1945) 80–83.
 - [34] D. P. Rodgers, Improvements in Multiprocessor System Design, *ACM SIGARCH Computer Architecture News* 13 (3) (1985) 225–231.
 - [35] C. H. C. Duarte, C. Talcott, Clara: An Actor Language for High-Performance Distributed Computing, in: *Proc. 12th Symposium on Computer Architecture and High-Performance Computing (SBAC-PAD 2000)*, São Pedro, Brazil, 20–37, 2000.
 - [36] Y. Yang, B. C. d. S. Oliveira, Unifying Typing and Subtyping, *Proc. 23th Int. ACM Symposium on Object-Oriented Programming Languages (OOPSLA 2017)*, Vancouver, Canada 1 (2) (2017) 47:1–47:26.
 - [37] A. Rastogi, et al., Safe and Efficient Gradual Typing for TypeScript, in: *Proc. 42nd Int. ACM Symposium on Principles of Programming Languages (POPL 2015)*, Mumbai, India, ACM, 167–180, 2015.
 - [38] R. Collobert, S. Bengio, J. Mariethoz, Torch: a modular machine learning software library, Tech. Rep. IDIAP-RR-46-2002, IDIAP Research Institute, 2002.
 - [39] A. Skyrme, N. Rodriguez, R. Ierusalimschy, Scripting Multiple CPUs with Safe Data Sharing, *IEEE Software* 31 (5) (2014) 44–51.
 - [40] F. H. de Carvalho Junior, C. A. Rezende, J. de C. Silva, W. G. Al-Alam, J. M. U. de Alencar, Contextual Abstraction in a Type System for Component-based High-Performance Computing Platforms, *Science of Computer Programming* 132 (Part 1) (2016) 96–128.
 - [41] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A Fresh Approach to Numerical Computing, *SIAM Review* 59 (2017) 65–98.
 - [42] C. Lattner, V. Adve, LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation, in: *Proc. Int. Symposium on Code Generation and Optimization: Feedback-directed and Runtime Optimization (CGO 2004)*, IEEE, Palo Alto, California, 75–86, 2004.
 - [43] M. Lu, et al., MrPhi: An Optimized MapReduce Framework on Intel Xeon Phi Coprocessors, *IEEE Transactions on Parallel and Distributed Systems* 26 (11) (2015) 3066–3078.
 - [44] J. Li, X. Ma, S. Yoginath, G. Kora, N. F. Samatova, Transparent Runtime Parallelization of the R Scripting Language, *Journal of Parallel and Distributed Computing* 71 (2011) 157–168.